Deep Q-Networks and the Cross-Entropy Method

From tables to function approximation; from values to policy search

---

ESE 561, Theory of Artificial Intelligence

Where we are

So far: one number per state or pair (toy MDP, cliff).

Today: the state space is too large for a table.

1. approximate Q-learning, and why it breaks
2. experience replay and the target network: DQN
3. pixels, frame stacks, partial observability (POMDPs)
4. a different idea: the cross-entropy method

Notebooks

- `Lecture_DQN_CEM.ipynb`: theory + demos
- `DQN_in_class.ipynb`: DQN on CartPole, step by step

Approximate Q-learning

How many states does an Atari game have?

Observation: a 210×160 RGB screen, 8 bits per channel:

$$\#\text{screens} = 256^{210 \cdot 160 \cdot 3} = 2^{806,400}.$$

True state: the console's memory, 128 bytes of RAM:

$$\#\text{states} \leq 2^{128 \cdot 8} = 2^{1024}.$$

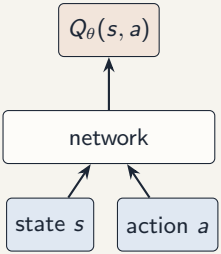
Far fewer, but still astronomically many, and we do not see the RAM.

Two ways out

- **Binarize** the state space: bins or hand-made features.
  too coarse: states we must tell apart look the same; too fine: every cell is visited once.
- **Approximate** Q with a function $Q_\theta(s, a)$, $\dim \theta \ll \#\text{states}$.
  similar inputs $\Rightarrow$ similar values: one update teaches many states (generalization).

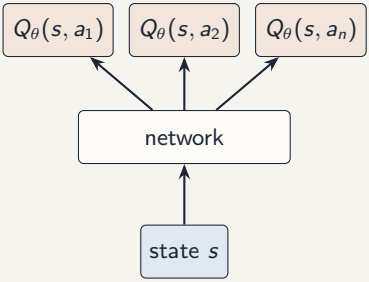
A table is a function approximator too: one-hot features $\phi(s, a) = e_{(s,a)}$.

Two architectures



given (s, a), predict one value

continuous or very many actions



given s, predict all $Q_\theta(s, \cdot)$ at once

one pass gives $\max_{a'}$ and ε-greedy: DQN

The last layer is linear (no activation): Q-values can be any real numbers.

Approximate Q-learning: regression onto a moving target

Target (as in tabular Q-learning, from the old values):

$$Y = r + \gamma\,(1-d)\max_{a'} Q_\theta(s', a')$$

Loss on one transition, with Y treated as a constant:

$$L(\theta) = \tfrac{1}{2}\big(\underbrace{Y}_{\text{const}} - Q_\theta(s,a)\big)^2$$

Gradient step:

$$\theta \leftarrow \theta + \alpha\,\big(Y - Q_\theta(s,a)\big)\nabla_\theta Q_\theta(s,a)$$

In code: compute Y under `torch.no_grad()`. A *semi-gradient* method: not the gradient of a fixed loss.

Same loss, three targets

Q-learning	$r + \gamma \max_{a'} Q(s', a')$
SARSA	$r + \gamma\, Q(s', a')$
Exp. SARSA	$r + \gamma\, \mathbb{E}_{a' \sim \pi} Q(s', a')$

One-hot check

$\nabla_\theta Q_\theta(s,a) = e_{(s,a)}$: only the visited entry moves, by $\alpha(Y - Q)$: tabular Q-learning.

Why it can diverge

The smallest example (Sutton & Barto, 11.2)



Linear $V_\theta(s) = \theta x_s$ with $x_1 = 1$, $x_2 = 2$. True values: 0, and $\theta = 0$ represents them exactly. TD on $s_1 \to s_2$:

$$\theta \leftarrow \big(1 + \alpha(2\gamma - 1)\big)\theta.$$

Updating only this transition (off-policy data), $\gamma = 0.9$: $\theta = 1 \to 4.7 \to 21.7 \to 101$ after 20, 40, 60 updates.

The deadly triad

1. function approximation
2. bootstrapping
3. off-policy data

together can diverge. Q-learning with a network has all three.

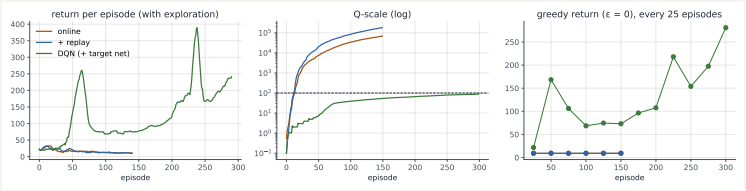
On CartPole: online Q-learning with an MLP drives the Q-values to $\sim 7 \cdot 10^4$; the true maximum is $1/(1-\gamma) = 100$.

Correlated data and experience replay

How bad is it if the agent spends 1000 steps in one corner?

Online learning feeds the network transitions in the order they happen.

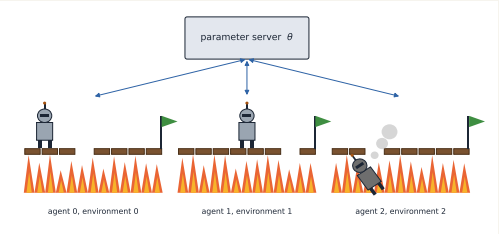
- **Not i.i.d.:** consecutive states are nearly identical (CartPole: 4× closer than random pairs, autocorrelation 0.85–0.95). 64 consecutive steps ≈ one sample repeated.
- **Forgetting:** 1000 steps in one corner overwrite what the network knew about the rest of the world.
- **Drops on the learning curve:** bad values → agent goes elsewhere → forgets the old region.
- **Waste:** each transition is used once.



online Q-learning, + replay, DQN (replay + target network) on CartPole

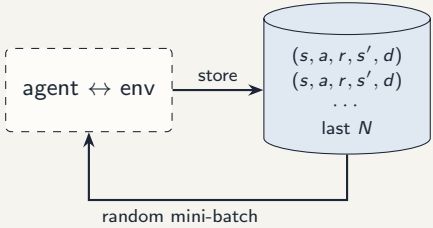
Two fixes: many agents, or a memory

Parallel agents with shared θ



- they explore different parts of the world at once
- works for on-policy methods (A3C, PPO)
- needs many environments. Q: a real robot car?

Experience replay (Lin, 1992; DQN 2013)



- mini-batches mix many episodes: close to i.i.d.
- old regions keep being rehearsed
- each transition is reused; older ones come from weaker, older policies

Replay, quantitatively

Ring buffer of the last N transitions. Every k environment steps: sample B uniformly, one gradient step.

How often is a transition used? It stays N steps, during N/k updates, each picking it with probability B/N:

$$\mathbb{E}[\#\text{uses}] = \frac{N}{k} \cdot \frac{B}{N} = \frac{B}{k}.$$

Independent of N: the capacity sets *how old* the data may be, the *replay ratio* B/k how often each sample is reused.

	Atari (2015)	CartPole
capacity N	10^6	$5 \cdot 10^4$
mini-batch B	32	64
steps per update k	4	1
uses per transition	8	64
warm-up steps	$5 \cdot 10^4$	64

Stored: transitions, never targets

Y is recomputed each time a transition is sampled, with the current target network: a stored Y would use old, worse values.

Quiz: which of these fail with experience replay?

You approximate $Q(s, a)$ with a network and train on a replay buffer.

Q-learning	CEM
SARSA	Expected SARSA

Quiz: which of these fail with experience replay?

You approximate $Q(s, a)$ with a network and train on a replay buffer.

Q-learning **CEM**

SARSA **Expected SARSA**

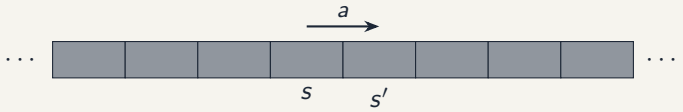
The agent trains off-policy on older versions of itself.

- **Q-learning:** the target $r + \gamma \max_{a'} Q(s', a')$ is valid for any (s, a), whoever chose a.
- **Expected SARSA:** averages over the *current* $\pi(\cdot|s')$; the stored a' is not used.
- **SARSA:** needs a' from the current policy; a stored a' came from an old one.
- **CEM:** selects elites among episodes of the *current* policy distribution.

On-policy methods: no (or a small) buffer; use parallel environments instead.

The moving target

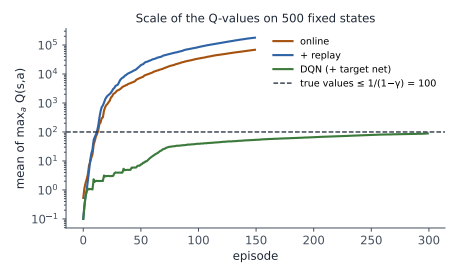
The target runs away



Consecutive states look alike, so $Q_\theta(s, \cdot)$ and $Q_\theta(s', \cdot)$ are strongly correlated through the shared weights.

Raising $Q_\theta(s, a)$ towards Y also raises $\max_{a'} Q_\theta(s', a')$, i.e. Y itself: the target moves with every step, and can run away.

Replay fixes the *data*, not the *target*: replay alone still diverges on CartPole (Q-scale $\sim 2 \cdot 10^5$, true values ≤ 100).



Target network

Idea: compute the target with a frozen copy θ^-:

$$L(\theta) = \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\Big[\ell\big(\underbrace{r + \gamma(1-d)\max_{a'} Q_{\theta^-}(s',a')}_{\text{const}} - Q_\theta(s,a)\big)\Big]$$

Hard update: every C steps, $\theta^- \leftarrow \theta$.
Soft update: every step, $\theta^- \leftarrow (1-\tau)\theta^- + \tau\theta$, $\tau \approx 0.005$.

Between refreshes: ordinary regression onto the fixed function TQ_{θ^-}. Each refresh $\approx$ one step of value iteration (*fitted Q-iteration*). Slow target, fast network: two time scales.

ℓ: Huber loss, $\frac{1}{2}x^2$ for $|x| \le 1$, $|x| - \frac{1}{2}$ beyond (bounded gradient). d: *terminated* only, not *truncated* by a time limit.

How long is the target frozen?

setting	refresh period C
Nature DQN, Atari	10,000 updates = 40,000 agent steps (1 update per 4 steps)
our CartPole	200 updates = 200 steps
soft update, $\tau = 0.005$	θ^- averages the last $\approx 1/\tau = 200$ steps

Rule of thumb

- long enough that Q_θ nearly fits the frozen target
- short enough for many refreshes: $\gg 1/(1-\gamma)$ Atari: $12.5 \cdot 10^6/10^4 \approx 1250$; CartPole: $37{,}000/200 \approx 185$

Too short: $C = 1$

no target network at all; on CartPole the Q-scale reaches $6.6 \cdot 10^5$ (true ≤ 100).

Too long: $C = 2000$

each refresh is one value-iteration step, and values need $\sim 1/(1-\gamma) = 100$ of them; after 300 episodes only ~ 15 refreshes, Q-scale still 2: almost nothing learned.

And the Huber loss

gradient $= \mathrm{clip}(\delta, -1, 1)$: one transition with a huge TD error (early training, right after a refresh, a rare big reward) cannot jolt the shared weights.

DQN (Mnih et al., 2013, 2015)

1. initialize Q_θ; $\theta^- \leftarrow \theta$; empty buffer $\mathcal{D}$
2. each step: *a* ε-greedy from $Q_\theta(s, \cdot)$; observe r, s', d; store (s, a, r, s', d)
3. every k steps: mini-batch from $\mathcal{D}$; targets with θ^-; one gradient step on the Huber loss
4. every C updates: $\theta^- \leftarrow \theta$

The 2013 workshop paper had replay only; the target network came with the 2015 *Nature* paper.

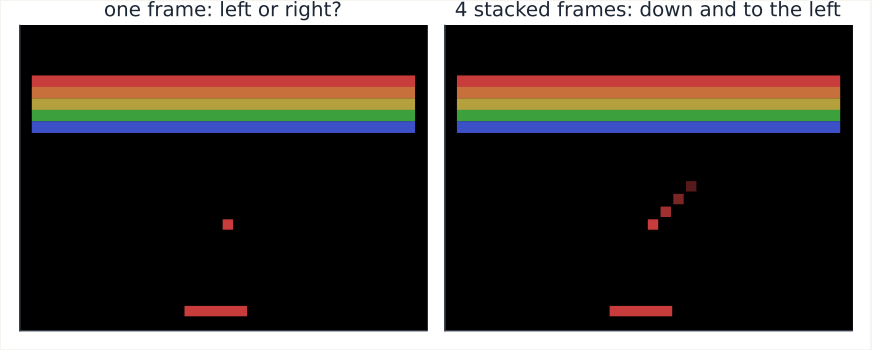
Nature DQN, Atari

input	last 4 frames, 84×84
replay capacity	10^6
mini-batch	32, every 4 steps
target refresh C	10,000 updates
γ	0.99
ε	$1 \to 0.1$ over 10^6 frames
optimizer	RMSProp, $2.5 \cdot 10^{-4}$

Same network and hyperparameters for 49 games.

Pixels and partial observability

Left or right?



One frame shows *where* the ball is, not *where it is going*: two different states give the same observation, so any function of one frame must treat them the same. The observation o_t is not the state s_t: a partially observable MDP.

Unknown model versus hidden state

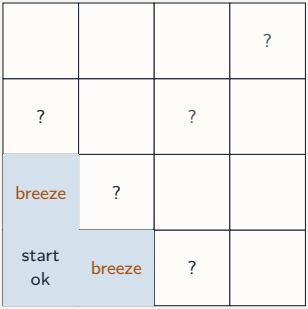
	state observed	state hidden
model known	MDP: DP, value iteration (toy MDP)	POMDP planning (probabilistic Wumpus)
model unknown	model-free RL: Q-learning, DQN on CartPole's 4 numbers	Atari from pixels, poker, robots

POMDP: an MDP plus observations $o \sim O(o \mid s)$; the agent sees o, not s. Different states can look the same.

The test: does the missing information change from episode to episode?

- **Cliff, FrozenLake:** the cliff and the holes are the same every episode. They are part of the dynamics $p(s'|s, a)$, learned into Q. The agent sees its cell: an **MDP** (unknown model; slippery ice = stochastic, not hidden).
- **Wumpus:** pits and Wumpus are placed anew every game and not observed: hidden state, a **POMDP**.
- **One Atari frame:** position visible, velocity hidden.

Wumpus: why memory is needed, and what it should hold



pit prior 0.2; risk of (2,2) ≈ 0.86, of (1,3), (3,1) ≈ 0.31

The percept in the current cell ("breeze") does not tell where the pit is; combining percepts over time does. The best action depends on the history.

- **Belief state:** $b(s) = P(s_t \mid \text{everything seen})$, the worlds still consistent, weighted. Beliefs are Markov: a POMDP is an MDP over beliefs (needs the model; POMCP = MCTS over sampled worlds).
- **Information state:** use as state everything perceived so far (breeze/stench per visited cell, position, arrow). Sufficient for the future, hence Markov: plain Q-learning/DQN works on it.
- **Learned memory:** an RNN over the percept sequence (DRQN) learns its own summary.
- **Frame stack** = a truncated history: the last n observations only.

The n-gram trick: stack the last frames

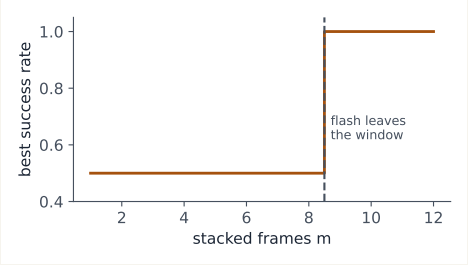
Idea: $s_t \neq o_t$, but

$$s_t \approx \left(o_{t-n}, a_{t-n}, \ldots, o_{t-1}, a_{t-1}, o_t\right)$$

(DQN: the last 4 frames, no actions).

- an n-th order Markov assumption
- **works** for velocities, accelerations, short timers (finite differences)
- **fails** for anything older than n frames
- **impractical** for large n

Toy: a flash at $t = 0$ tells which door is correct; the choice is made 8 frames later.



The Atari pipeline, and the price of a fixed window

step	why
frame skip 4	15 decisions/s; 4× cheaper
max of 2 frames	*flicker*: sprites drawn on alternate frames
gray, 84×84	28,224 inputs instead of 100,800
stack 4	velocities (approximately Markov)
clip r to $[-1, 1]$	one learning rate for all games

- **memory horizon** $4 \times 4 = 16$ raw frames $\approx 0.27\,\text{s}$
- **flicker** $\times$ **skip**: without the max, an object can be invisible in all 4 frames
- **episode boundaries**: a stack must not mix two games (pad)
- **replay memory** (10^6 transitions): stacked s, s': 56 GB; each frame once, stacks rebuilt from indices: 7 GB

Learned memory instead of a window

DRQN (Hausknecht & Stone, 2015): one frame per step into an LSTM. R2D2 (2019): recurrent replay of sequences with a "burn-in". → RNNs are our next topic.

The Atari network

layer	output	parameters
input	$4 \times 84 \times 84$	–
conv 32, 8×8, stride 4	$32 \times 20 \times 20$	8,224
conv 64, 4×4, stride 2	$64 \times 9 \times 9$	32,832
conv 64, 3×3, stride 1	$64 \times 7 \times 7$	36,928
dense 512	512	1,606,144
output	18 actions	9,234

Output size: $\lfloor (W - K + 2P)/S \rfloor + 1$.

Total 1.7 M parameters; one dense layer of 512 on the raw input alone: 14.5 M.

Frames enter as 4 *channels*: only the first layer depends on the window size, and the network knows nothing outside it.

The cross-entropy method

Back to the objective

The easy way: $\mathbb{E}_\pi[R]$ is the expected total reward per episode of the policy π.

The hard way:

$$\mathbb{E}_{s_0 \sim p_0} \mathbb{E}_{a_0 \sim \pi(\cdot|s_0)} \mathbb{E}_{s_1, r_0 \sim p(\cdot|s_0, a_0)} \cdots \left[r_0 + r_1 + \cdots + r_T \right]$$

Find π with $\mathbb{E}_\pi[R] \to \max$.

DQN: learn values, act greedily. Why not search over policies directly?

General idea

play a few episodes

↓

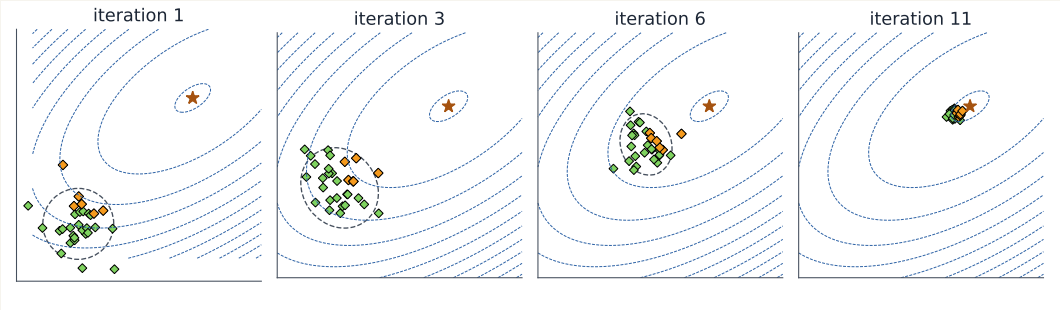
update the policy

↓

repeat

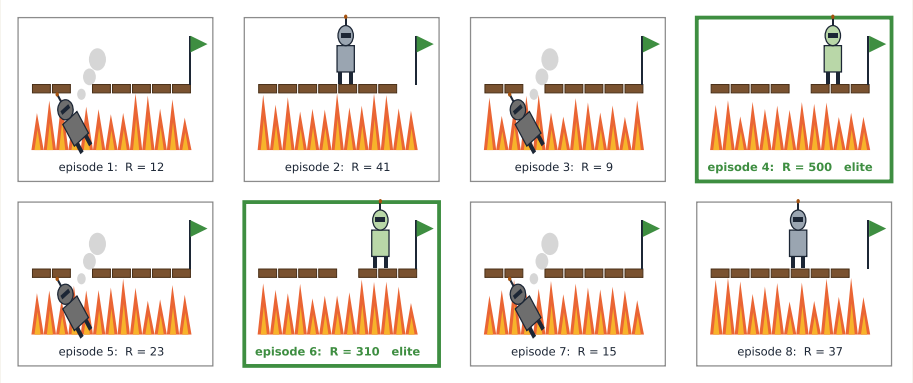
Cross-entropy method

Initialize the policy. Repeat: **(1)** sample N (100) episodes; **(2)** keep the M (25) best, the elite episodes; **(3)** change the policy so that it prefers the actions of the elites.



Search in a 2-D parameter space: samples (green), elites (orange), the sampling Gaussian (dashed), the optimum (star). The Gaussian is refit to the elites at every iteration.

One CEM iteration: play many episodes, keep the elites



The same policy plays N episodes (in parallel, if we can). Only the order of the returns matters: the top fraction (here 2 of 8) are the **elites**; the policy is refit to imitate them, the other episodes are discarded.

Tabular CEM (Taxi, toy MDP): the update and a worked example

Counts in the elite episodes of iteration k:

$$N_k(s,a) = \#\{\text{times } a \text{ was taken in } s\},$$
$$N_k(s) = \textstyle\sum_a N_k(s,a) \quad \text{(visits to } s\text{)}.$$

Maximum-likelihood fit, one categorical per state:

$$\hat{\pi}_k(a \mid s) = \frac{N_k(s,a)}{N_k(s)}$$

Smoothed update, $0 < \lambda \leq 1$:

$$\pi_{k+1}(a \mid s) = \lambda\, \hat{\pi}_k(a \mid s) + (1-\lambda)\, \pi_k(a \mid s),$$

and $\pi_{k+1}(\cdot|s) = \pi_k(\cdot|s)$ if $N_k(s) = 0$ (no elite visited s).

$\lambda = 1$: plain CEM; an action dropped once is never tried again. $\lambda < 1$ keeps probability on every action.

Toy MDP: 50 episodes, 10 elites, $\lambda = 0.5$, start $\pi(\text{go}|A) = \pi(\text{cash}|B) = 0.5$.

Iteration 1: elites took go in A 10 times out of 22, cash in B 10 out of 10:

$$\pi_1(\text{go}|A) = 0.5 \cdot \tfrac{10}{22} + 0.5 \cdot 0.5 = 0.48$$
$$\pi_1(\text{cash}|B) = 0.5 \cdot \tfrac{10}{10} + 0.5 \cdot 0.5 = 0.75$$

iter	go in A	cash in B	new π
1	10 / 22	10 / 10	(0.48, 0.75)
2	10 / 10	10 / 10	(0.74, 0.88)
3	10 / 10	10 / 10	(0.87, 0.94)
4	10 / 10	10 / 10	(0.93, 0.97)

Deep CEM (`deep_crossentropy_method.ipynb`)

Large state space: $\pi_W(a|s)$ is a network; we cannot set $\pi(a|s)$ explicitly. Instead: maximize the likelihood of elite actions,

$$W \leftarrow W + \alpha \nabla_W \sum_{(s_i,a_i)\in \text{Elite}} \log \pi_W(a_i|s_i) :$$

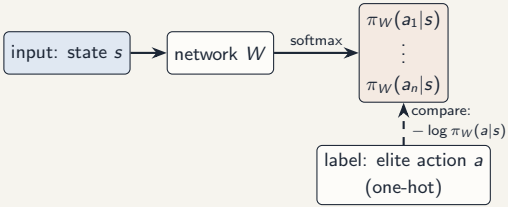
classification with cross-entropy loss, `nn.fit(elite_states, elite_actions)`.

Discrete actions (CartPole, toy MDP, Taxi): a classifier, as above. **Continuous actions:** $\pi_W(a|s) = \mathcal{N}(\mu_W(s), \sigma^2)$. Maximizing $\log \pi$ = regression onto the elite actions (`MLPRegressor`). Almost nothing changes.

Tricks

- reuse elites from the last 3–5 iterations
- entropy bonus / noise floor against premature convergence
- parallel sampling; RNN policy for POMDPs

Deep CEM: input, output, target, loss



- **input** s: CartPole's 4 numbers; toy MDP: one-hot of A/B
- **output**: one probability per action
- **target (label)**: the action taken in s in an *elite* episode
- **loss**: cross-entropy over all elite pairs

$$L(W) = -\sum_{(s_i,a_i)\in \text{Elite}} \log \pi_W(a_i \mid s_i)$$

Toy example. The elite episode $A \xrightarrow{\text{go}} B \xrightarrow{\text{cash}} G$ becomes two rows of a training set:

input s	label a
$A = (1,0)$	go
$B = (0,1)$	cash

The return is *not* in the loss: it only decides which episodes enter the training set. A non-elite episode contributes nothing.

Continuous actions: the output is a mean $\mu_W(s)$ of a Gaussian, and $-\log \pi_W(a|s) = \frac{(a-\mu_W(s))^2}{2\sigma^2} + \text{const}$: regression onto the elite actions.

Why policy gradients?

We want $\nabla_\theta J$ for $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$. Why not backpropagate through the episode?

- the environment $p(s'|s,a)$ is unknown, a simulator or the real world: no gradient through it;
- actions are *sampled*, often discrete: $\theta \mapsto$ trajectory is not differentiable;
- the reward may be a game score, a human rating: not differentiable either.

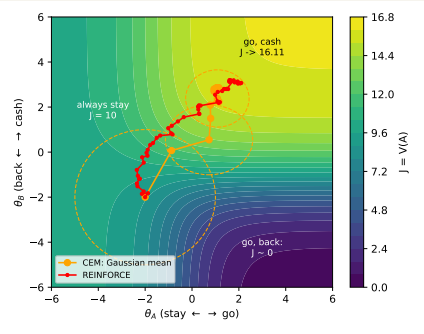
But θ enters only through the probability of the trajectory:

$$J = \sum_\tau p_\theta(\tau) R(\tau),$$

$$\nabla J = \mathbb{E}\big[R \nabla \log p_\theta(\tau)\big] = \mathbb{E}\Big[R \sum_t \nabla \log \pi_\theta(a_t|s_t)\Big]$$

(the dynamics drop out of $\nabla \log p_\theta$). Differentiate the policy, not the world.

Why not just CEM? It keeps 20% of the data with 0/1 weights, optimizes $P(R \geq \gamma)$ rather than $\mathbb{E}R$, and parameter-space CEM fails in high dimension.



Toy MDP, exact $J(\theta_A, \theta_B) = V(A)$: CEM moves a Gaussian cloud (2,250 episodes), REINFORCE a noisy gradient path (600 episodes); both reach $J \approx 15.6$–15.8.

Why "cross-entropy"?

Rubinstein (1997), rare-event simulation. Sampling density $f(\theta; v)$, elite threshold γ. The ideal next density keeps only good samples:

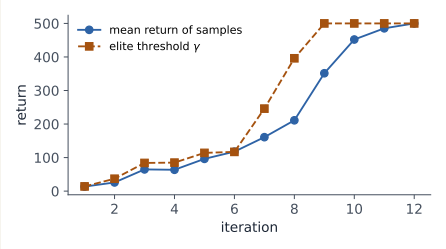
$$g^*(\theta) \propto \mathbf{1}\{J(\theta) \geq \gamma\}\, f(\theta; v).$$

Choose the closest member of the family:

$$\min_{v'} \mathrm{KL}(g^* \| f_{v'}) \iff \max_{v'} \mathbb{E}_v\left[\mathbf{1}\{J \geq \gamma\} \log f(\theta; v')\right]$$

$$\approx \max_{v'} \sum_{i \in \text{elite}} \log f(\theta_i; v') :$$

maximum likelihood on the elites. Gaussian: elite mean and variance. Policy network: the classifier fit above.



CartPole, linear policy $\theta \in \mathbb{R}^5$, $N = 50$, 20% elites: return 500 after 12 iterations, about a second of compute.

From CEM to policy gradients

Deep CEM is a gradient step on $\log \pi$ with a $0/1$ weight per episode:

$$\Delta W \propto \sum_i \mathbf{1}\{R_i \geq \gamma\} \sum_t \nabla_W \log \pi_W(a_t^i | s_t^i).$$

REINFORCE (Williams, 1992) uses the return itself, minus a baseline:

$$\Delta W \propto \sum_i (R_i - b) \sum_t \nabla_W \log \pi_W(a_t^i | s_t^i),$$

an unbiased estimate of $\nabla_W \mathbb{E}_\pi[R]$ that uses *all* episodes.

Replace $R_i - b$ by a learned value estimate: **actor–critic**, with TD learning back inside.

With a learned reward model as R: how LLMs are fine-tuned from human feedback (**RLHF**). We will meet it again.

Summary

	DQN	CEM
learns	$Q_\theta(s, a)$; acts greedily	a distribution over policies
uses	every transition, many times	one return per episode
needs	Bellman equation, Markov state	only a way to run episodes
data	off-policy: replay buffer	on-policy: fresh episodes
stabilizers	replay (data), target network (target), Huber	elite fraction, noise floor, smoothing
fails when	deadly triad; non-Markov input (fixed frame window)	many parameters; noisy returns; sparse rewards